

Linux

WPC

2026 年 4 月 18 日

目录

1	目录与文件	3
1.1	目录	3
1.2	文件	4
1.3	文件编辑	5
1.3.1	vim	6
1.4	压缩	7
2	Shell 编辑	8
2.1	快捷键	8
2.2	通配符	8
2.3	常用命令	9
2.4	shell 扩展	9
2.5	正则表达式	10
2.6	历史记录	11
2.7	颜色	11
3	进程	13
3.1	进程相关信息	13
3.2	作业控制	14
3.3	信号	14
4	环境	16
4.1	用户	16
4.2	软件包安装	16
4.3	环境变量	17
4.4	网络	18

目录	2
5 存储设备	19
5.1 设备信息	19
5.2 文件系统	20
6 Shell 脚本	22
6.1 脚本格式	22
6.2 变量	22
6.3 语句	23
6.4 参数展开	23
7 系统启动与服务	25
7.1 cron	25
7.2 systemd	25
7.3 GRUB	26

1 目录与文件

1.1 目录

常见的 Linux 目录名均基于文件系统层级标准（filesystem hierarchy standar, FHS）。

目录	说明
/bin	系统引导与执行所必需的用户可执行程序（如 <code>ls</code> , <code>cp</code> ）。
/boot	启动相关文件：内核、initramfs、引导程序配置等。
/dev	设备文件和特殊文件（由内核和 udev 管理）。
/etc	系统范围的配置文件。
/home	普通用户的家目录。
/lib	系统核心程序运行所需的共享库。
/lost+found	用于文件系统在崩溃或错误后的部分恢复，除非系统发生严重问题，否则该目录为空。
/media	可移动介质的挂载点（如 U 盘、光驱）。
/mnt	临时挂载点，管理员手动挂载时常用。
/opt	第三方商业软件的安装位置。
/proc	内核与进程信息的伪文件系统（内存中生成）。
/root	root 用户的家目录。
/run	运行时数据（通常为 tmpfs，重启后清空）。
/sbin	系统管理相关的必需命令（如 <code>mount</code> , <code>fsck</code> ）。
/sys	设备与内核对象的伪文件系统（sysfs）。
/tmp	临时文件目录，重启后可能被清理。
/usr	用户空间程序和数据的层级根。
/usr/bin	大量用户可执行程序。
/usr/local	本地安装的软件及数据前缀（通常通过源代码编译安装）。
/usr/sbin	系统管理相关的命令。
/usr/share	/usr/bin 中的程序所用到的共享数据，包括默认配置文件、图标、声音文件等。
/usr/share/doc	系统中安装的大部分软件包自带文档。
/var	修改比较频繁的数据存放在这里，比如：缓存、数据库、spool、用户邮件 等。
/var/log	系统和应用程序的日志文件目录，比较有用的有 /var/log/messages 和 /var/log/syslog。

表 1.1: Linux 常见目录与说明

目录与文件操作

```
cd ~user          # 切换到指定用户的家目录
cd -              # 切换到上一个工作目录
cp -r src dst     # 递归复制目录
mv src dst        # 移动或重命名
rm -r dir         # 递归删除目录
mkdir -p a/b/c    # 递归创建目录
ln -s target link # 创建软链接
file foo          # 识别文件类型
rename -v 's/(.*) - (.*)\.(.*)/$2 - $1.$3/' *
rename -v 's/tol24.(.*)/$1/' *.swf          # 批量重命名（支持正则）
```

文件查找

```
locate filename          # 从数据库快速查找文件
stat file                # 显示文件详细元信息
find dir -type f -name "*.txt"      # 按类型和名称查找
find dir -type f -size +10M         # 查找大于10M的文件
find dir -type f -name "*.log" | xargs rm # 找到后批量删除
```

1.2 文件

类型	说明
-	普通文件。
d	目录。
l	链接文件, 对于链接文件，文件模式始终是虚设的 <code>rwxrwxrwx</code> 。
c	字符设备文件，比如终端和 <code>/dev/null</code> 。
b	块设备文件，比如硬盘和 DVD 设备。

表 1.2: 文件类型

文件模式共九个字符，分别定义了文件拥有者、文件所属组和其他用户对于文件的读、写、执行权限。

文件权限管理

```
chmod 755 file      # 修改文件权限
chown user file     # 修改文件所有者
chown user:group file # 同时修改拥有者和所属组
chgrp group file    # 修改文件所属组
```

权限可以用数字表示法（如 755）来设置，数字表示法中，每个数字是三个权限位的和，分别对应拥有者、所属组和其他用户。常用的有：7 (rwx)、6 (rw-)、5 (r-x)、4 (r-)、0 (—)，可以用 umask 查看文件创建时默认权限要屏蔽的位掩码。

模式	文件	目录
r	允许打开和读取文件内容	允许列出目录内容（需要执行属性）
w	允许修改文件内容	允许在目录中创建、删除和重命名文件（需要执行属性）
x	允许执行文件	允许进入目录

表 1.3: 文件权限

除了基本权限外，还有特殊权限：setuid、setgid 和粘滞位，分别显示在文件模式的用户执行位、组执行位和其他用户执行位上，用数字表示法时，特殊权限作为第四位前缀添加在普通三位权限之前，setuid 为 4，setgid 为 2，粘滞位为 1，例如 4755 表示设置了 setuid 位。

- **setuid**（可执行文件）：执行时以文件拥有者的权限运行。
- **setgid**（可执行文件）：执行时以文件所属组的权限运行。
- **setgid**（目录）：目录下新建的文件将继承该目录的所属组，一般共享目录会设置此位。
- **粘滞位**（目录）：只有文件拥有者、目录拥有者或超级用户才能删除或重命名该目录下的文件。

1.3 文件编辑

cat	sort	uniq	cut	paste	join
comm	diff	patch	tr	sed	aspell
nl	fold	fmt	rev	groff	seq

表 1.4: 文件操作命令

文件编辑常用命令

```
cut -d: -f1 /etc/passwd      # 以:为分隔符，提取第1列
paste file1 file2            # 按列合并两个文件
join file1 file2             # 根据公共字段连接两个文件
comm file1 file2             # 逐行比较两个已排序文件
diff file1 file2             # 比较两个文件的差异
patch file < file.patch      # 根据 diff 补丁更新文件
tr 'a-z' 'A-Z'              # 将小写字母转换为大写
rev file                     # 反转每行字符
expand file                  # 将 tab 转换为空格
sed 's/old/new/g' file      # 全局替换
sed -n '1,5p' file           # 只输出1到5行
sed -n '/pattern/p' file     # 只输出匹配行
grep -n -E 'regex' file      # 显示行号，使用扩展正则
grep -i 'pattern' file       # 忽略大小写
printf "%s\n" str1           # 格式化输出
aspell check file            # 拼写检查
seq 1 10                     # 生成 1 到 10 的数字序列
```

1.3.1 vim

如果你在 vi 中不清楚自己处于什么模式，可以通过两次 esc 来返回命令模式。

命令	说明
0	移动到行首
^	移动到行首第一个非空白字符
g_	移动到行尾最后一个非空白字符
\$	移动到行尾
w / b	向后/前移动一个单词，大写只以空格为界
e / ge	移动到单词结尾/向前移动到单词结尾
2G	移动到第二行
G	移动到最后一行
gg	移动到第一行

表 1.5: vim 常用移动键位

命令	说明
x	删除当前字符
2x	删除当前 2 个字符
dd	删除当前行
2dd	删除当前两行
dG	从当前行删到最后一行
d20G	从当前行删到 20 行
d\$	从当前字符删到行尾
d0	从当前字符删到行首

表 1.6: 删除键位（d 剪切，y 复制，p 粘贴）

- 行内搜索：fa 移动到行内下一个 a，ta 移动到 a 前一个字符；；下一个，，上一个。

- 全局搜索：/ 进行全局搜索，n 重复上次搜索。
- 行内替换：:s/Line/line/g, s 是替换命令，g 代表找到的结果全部替换。
- 全局替换：命令为%s/Line/line/g, %代表搜索整个文件，也可以写成1,\$，代表从第 1 行到最后一行。

vim 其他常用命令

```
.          # 重复上次操作
u          # 撤销, Ctrl+r 重做
I          # 跳到行首并进入插入模式
A          # 跳到行尾并进入插入模式
Ctrl+o    # 插入模式下临时执行一次普通模式命令
ggVG      # 全选 (gg到首行, V进入行选模式, G到末行)
:r foo.txt # 将 foo.txt 内容插入当前光标下方
```

1.4 压缩

压缩与归档

```
gzip file          # 压缩为 file.gz (原文件被替换)
gunzip file.gz     # 解压 .gz
bzip2 file         # 压缩为 file.bz2 (压缩率更高)
bunzip2 file.bz2   # 解压 .bz2
zcat file.gz       # 不解压直接查看内容
zip archive.zip file1 file2 # 打包压缩
unzip archive.zip   # 解压 .zip
```

2 Shell 编辑

2.1 快捷键

快捷键	说明
Ctrl + T	打开新标签页。
Ctrl + W	关闭标签页。
Ctrl + L	跳转地址栏。
Ctrl + Tab	切换标签页。

表 2.1: 浏览器 常用快捷键

快捷键	说明
Ctrl + a	移动光标到行首。
Ctrl + e	移动光标到行尾。
Alt + f	将光标向前移动一个单词。
Alt + b	将光标向后移动一个单词。
Ctrl + k(kill)	剪切从光标位置到行尾的内容。
Ctrl + y(yank)	粘贴最近删除的内容。
Ctrl + l	清屏，相当于执行 clear 命令。
Ctrl + r	反向搜索历史命令。

表 2.2: Bash 常用快捷键

2.2 通配符

注意通配符是 shell 处理的，而不是命令处理的。*并不会匹配以.开头的文件，要匹配这些文件需要使用.[!]*。

通配符	说明
*	匹配任意多个字符。
?	匹配任意单个字符。
[characters]	匹配方括号内的任意单个字符。
[!characters]	匹配不在方括号内的任意单个字符。
[[:class:]]	匹配属于字符类的任意单个字符，字符类有 alnum、alpha、digit、lower、upper 等。

表 2.3: 常见通配符与说明

2.3 常用命令

alias	bg	bind	builtin	cd	command	compgen
complete	disown	echo	enable	exec	exit	export
fc	fg	hash	help	history	jobs	kill
logout	printf	pwd	type	ulimit	umask	wait

表 2.4: Bash 内置命令

- 查找命令：apropos、dpkg -S。
- 帮助命令：help、info、man、whatis、whereis、which。
- 编辑命令：wc、head、tail、cut、grep、sort、uniq。
- 输出重定向：> file 2>&1 相当于&> file，|管道将命令的标准输出传给下一个命令的标准输入。
- 输入重定向：cat 读取文件到标准输出，tee 读取标准输入并将其同时写入标准输出和文件中，xargs 将标准输入转换为命令的参数。

常用命令示例

```
date                # 显示当前日期时间
wc -l file          # 统计行数 (-w词数 -c字节数)
head -n 5 file      # 显示前5行
tail -n 5 file      # 显示后5行
tail -f file        # 实时跟踪文件末尾 (适合看日志)
ls /bin /usr/bin | sort | uniq | less # 管道：合并排序去重分页
export PATH=~/bin:$PATH # 将 ~/bin 追加到 PATH
```

2.4 shell 扩展

参数扩展、算术扩展和命令替换仍然会在双引号中进行，但单引号中所有扩展均不能用。

- 参数扩展：\${var}表示变量的值，\${#var}表示变量值的长度。
- 算术扩展：\$((expression)), 如echo \$((3 + 5))。
- 命令替换：\$(command), 如echo "Today is \$(date)"。
- 进程替换：<(cmd) 可以将命令的输出当成文件使用。

- 路径名扩展: ~/表示用户主目录, ~+表示当前工作目录, ~-表示上一个工作目录。
- 花括号扩展: {1..5}表示生成 1 到 5 的数字序列, {a..e}表示生成 a 到 e 的字母序列。
- 历史扩展: !!表示上一条命令, !n 表示第 n 条命令, !string 表示最近一条以 string 开头的命令, !?string 表示最近一条包含 string 的命令。如果你需要记录 shell 对话, 可以使用script命令。

2.5 正则表达式

正则表达式中存在元字符 (metacharacter) : . ? + * { } ^ \$ [] - () | \

POSIX 字符类可用于方括号表达式中, 常用的有: [:upper:] (大写字母)、[:lower:] (小写字母)、[:alpha:] (字母)、[:digit:] (数字)、[:alnum:] (字母和数字)、[:space:] (空白字符)。例如 [:upper:]* 匹配以大写字母开头的任意字符串。

简写字符类 (大写为补集)			
\d	[:digit:]	# 数字	\D 非数字
\s	[:space:]	# 空白字符	\S 非空白
\l	[:lower:]	# 小写字母	\L 非小写
\u	[:upper:]	# 大写字母	\U 非大写
\w	[_:alnum:]	# 单词字符	\W 非单词字符
\b	单词边界		\B 非单词边界

锚点与量词	
^	行首 (在 [...] 中为取反)
\$	行尾
~\$	空行
?	0 或 1 次
*	0 或多次
+	1 或多次
{n}	恰好 n 次
{n,}	至少 n 次
{n,m}	n 到 m 次 (含两端)

2.6 历史记录

ctrl+r 搜索历史记录，esc 退出搜索。

```
history

history 10      # 显示最近 10 条历史记录
history -c      # 清空历史记录
!-3:p          # 打印倒数第 3 条命令（不执行）
rm !$          # !$ 表示上一条命令的最后一个参数，先 ls 再 rm !$
^jg^jpg        # 将上一条命令中的 jg 替换为 jpg 并执行（只替换首次出现）
!!:s/jg/jpg/    # 同上，替换语法的另一种写法
```

2.7 颜色

Shell 中也能显示颜色，分为文本颜色和背景颜色，关闭颜色的代码是\033[0m。

转义代码	文本颜色
\033[0;30m	黑色 (black)
\033[0;31m	红色 (red)
\033[0;32m	绿色 (green)
\033[0;33m	棕色 (brown)
\033[0;34m	蓝色 (blue)
\033[0;35m	紫色 (magenta)
\033[0;36m	青色 (cyan)
\033[0;37m	浅灰色 (light grey)

表 2.5: 常用文本颜色代码

转义代码	文本颜色
\033[1;30m	深灰色 (deep grey)
\033[1;31m	浅红色 (light red)
\033[1;32m	浅绿色 (light green)
\033[1;33m	黄色 (yellow)
\033[1;34m	浅蓝色 (light blue)
\033[1;35m	浅紫色 (light magenta)
\033[1;36m	浅青色 (light cyan)
\033[1;37m	白色 (white)

表 2.6: 高亮文本颜色代码

转义代码	背景颜色
\033[40m	黑色
\033[41m	红色
\033[42m	绿色
\033[43m	棕色
\033[44m	蓝色
\033[45m	紫色
\033[46m	青色
\033[47m	浅灰色

表 2.7: 背景颜色代码

3 进程

3.1 进程相关信息

进程有进程 ID (Process ID, PID), init 进程的 PID 始终为 1, 和文件类似, 进程也有拥有者、用户 ID 和有效用户 ID 等。

进程状态	说明
R	运行或可运行状态 (正在运行或在运行队列中)。
S	休眠状态 (等待某个事件完成, 比如按键按下或网络包到来)。
D	不可中断的休眠状态 (通常是 IO)。
T	停止状态 (被信号停止)。
Z	僵尸状态 (进程已终止, 但其父进程尚未调用 wait() 获取其状态)。
<	高优先级进程。
N (nice)	低优先级进程。
L	进程有锁定在内存的页面。
l	进程有多线程。
+	进程在前台运行。

表 3.1: 常见进程状态

进程查看

ps aux

显示所有用户的所有进程

top

动态显示进程信息 (q退出, k发送信号)

列名	说明
USER	进程所有者的用户名。
%CPU	CPU 占用率。
%MEM	内存占用率。
VSZ	虚拟内存大小 (以 KB 为单位)。
RSS	驻留集大小, 物理内存大小 (以 KB 为单位)。
TTY	进程的控制终端, ?表示没有控制终端。
START	进程启动时间。
TIME	进程使用的 CPU 时间总和。

表 3.2: ps 信息字段说明

信息	说明
up 6:30	运行时间 (uptime)，自上次重启之后的时间总和 (6.5h)。
1 user	当前登录用户数。
load average: 0.07, 0.02, 0.00	平均负载，分别为过去 1 分钟、5 分钟和 15 分钟的平均负载。
0.7 us, 0.3 sy	分别是用户进程和系统（内核）进程占用 CPU 百分比。
0.0 ni	用户进程空间内低优先级进程占用 CPU 百分比。
99.0 id, 0.0 wa	分别是 CPU 空闲和等待 IO 百分比。
0.0 hi, 0.0 si	分别是硬中断和软中断占用 CPU 百分比。
0.0 st	被虚拟机偷取的 CPU 百分比。

表 3.3: top 总体状态说明

3.2 作业控制

- Ctrl + Z：暂停当前前台进程，并将其放入后台。
- jobs：查看当前用户的所有作业。
- fg %n：将编号为 n 的后台作业调回前台。
- bg %n：将编号为 n 的后台作业继续在后台运行。
- disown %n：将编号为 n 的后台作业移出作业列表，退出 shell 时不会终止该作业。
- nohup command &：在后台运行 command，即使退出 shell，command 仍会继续运行。

3.3 信号

kill 和 pkill 命令发送信号给指定进程，前者需要 pid，后者可以用程序名，默认发送 TERM 信号 (15)，当使用 ctrl + c 终止前台进程时，实际上是向该进程发送 INT 信号 (2)，当使用 ctrl + z 暂停前台进程时，实际上是向该进程发送 TSTP 信号 (20)。

信号	说明
HUP (1)	终端断开连接时发送给前台进程。
INT (2)	中断进程，通常由 Ctrl + C 触发。
QUIT (3)	退出进程。
KILL (9)	内核会强制终止进程，信号其实不会发送给进程。
TERM (15)	请求进程终止，kill 默认发送信号。
CONT (18)	继续执行进程，bg 和 fg 命令会发送这个信号。
STOP (19)	停止进程，信号不会发向目标进程。
TSTP (20)	终端停止信号（terminal stop），通常由 Ctrl + Z 触发。

表 3.4: 常见信号及说明

进程控制

kill PID

发送 TERM 信号

killall NAME

按进程名发送信号

shutdown -h now

立即关机

poweroff

关机

reboot

重启

halt

停止系统

\$!

最近置入后台的进程 PID

wait \$pid

等待指定进程结束

{ cmd1; cmd2; }

在当前 shell 执行多命令

(cmd1; cmd2)

在子 shell 执行多命令

mkfifo pipe

创建命名管道

4 环境

4.1 用户

`id` 命令查看用户信息，用户定义在 `/etc/passwd` 文件中，属组定义在 `/etc/group` 文件中，一般来说，会为每个用户创建一个与用户同名的单成员属组。如果想切换用户可以使用 `su` 命令，可以使用 `sudo` 来以超级用户权限执行命令。

用户管理

```
useradd -m -G sudo -s /usr/bin/bash username
userdel -r username
usermod -aG sudo username
```

4.2 软件包安装

从源码安装时，`configure` 用于检测环境并生成 `Makefile`，再通过 `make` 编译，`make install` 安装到 `/usr/local/bin`。

从源码安装

```
tar xvf package.tar.gz -C /usr/local/src
./configure
make
make install
```

apt

```
apt install pkg
apt remove pkg      # 保留配置文件
apt purge pkg       # 删除配置文件
apt autoremove      # 删除不再需要的依赖包
apt show pkg        # 查看软件包详细信息
apt list --installed # 列出已安装的软件包
```


dpkg

dpkg -i pkg.deb # 安装本地 .deb 包
dpkg -l # 列出所有已安装的软件包
dpkg -s pkg # 查看软件包是否已安装及其状态
dpkg -L pkg # 列出软件包安装的所有文件
dpkg -S file # 查询文件由哪个软件包安装

4.3 环境变量

Shell 保存了两种数据类型：Shell 变量和环境变量。可以使用 printenv 命令查看环境变量、set 查看所有变量、alias 查看别名。

变量	说明
DISPLAY	屏幕名称，通常为:0，表示 X 服务器生成的第一个屏幕。
HOME	当前用户的主目录。
LANG	系统语言和区域设置。
OLDPWD	上一个工作目录。
PATH	由冒号分隔的目录列表，用于可执行文件的搜索。
PWD	当前工作目录。
SHELL	当前用户的默认 Shell。
TERM	终端类型。
USER	当前用户名。

表 4.1: 常见环境变量

环境变量操作

export PATH=~/bin:"\$PATH" # 将 ~/bin 追加到 PATH 前缀

Shell 会话通常分为两类：登录会话和非登录会话，二者在启动时所读取的配置文件有所不同。当修改了配置文件后，可以通过 source 或 . 命令使改动立即生效。登录会话在启动时需要用户输入用户名和密码进行身份验证，而非登录会话则无需凭证即可进入。通常，通过图形界面（GUI）启动的终端属于非登录会话。

文件	说明
/etc/profile	应用于所有用户的全局配置。
~/.bash_profile	用户个人的登录 Shell 配置文件。
~/.bash_login	如果 ~/.bash_profile 不存在，则读取该文件。
~/.profile	如果前两者都不存在，则读取该文件。

表 4.2: 登录 Shell 读取的配置文件

文件	说明
/etc/bash.bashrc	应用于所有用户的全局配置。
~/.bashrc	用户个人的非登录 Shell 配置文件, 通常会在 ~/.bash_profile 中被调用。

表 4.3: 非登录 Shell 读取的配置文件

4.4 网络

网络工具

tracert

host

追踪到目标主机的路由路径

netstat

-ie

显示网络接口信息 (同 ifconfig)

netstat

-r

显示路由表

rsync

-av --delete

src dest

同步目录 (-a归档 -v详细 --delete删除目标多余文件)

5 存储设备

5.1 设备信息

文件系统表 (file system tabl) /etc/fstab 列出会在系统引导时挂载的各种设备 (通常是硬盘分区)。

字段	说明
设备	文本标签或随机生成的通用识别码 (Universally Unique Identifier, UUID)。
挂载点	设备挂载到的目录。
文件系统类型	设备使用的文件系统类型, 如 ext4、swap。
挂载选项	挂载时的选项, 如 defaults、ro、noexec。
转储频率	用于 dump 命令的备份选项, 通常为 0 (不备份) 或 1。
自检顺序	用于 fsck 的文件系统检查顺序: 0=不检查, 1=先检查, 2=后检查。

表 5.1: /etc/fstab 文件字段

在 Linux 中, 可以使用 mount 命令挂载文件系统, 并通过 umount 命令卸载文件系统, 命令仅对当前会话有效, 如果需要开机自动挂载, 得在/etc/fstab 中写入内容。需要注意的是, 音频 CD 与常见的 CD-ROM 光盘不同, 音频 CD 本身不包含文件系统, 因此无法像普通数据光盘那样直接挂载。另外, 由于系统在读写设备时会使用缓冲区, 如果在数据尚未写回设备之前没有执行卸载操作, 直接移除或关闭设备, 就可能导致数据丢失或损坏。

设备	说明
/dev/lp*	并行端口打印机设备。
/dev/sd*	SCSI 磁盘及其分区。在现代 Linux 中, 内核将所有类似磁盘的设备 (如 SATA 硬盘、USB 闪存、数码相机等存储设备) 都视为 SCSI 设备。sda 表示第一个磁盘, sda1 是它的第一个分区, sdb 是第二个磁盘, 依此类推。
/dev/nvmeNn*	nvme 磁盘及其分区, N 是磁盘编号。
/dev/sr*	光驱设备 (CD-ROM/DVD)。
/dev/null	空设备, 写入数据会丢弃, 读取返回 EOF。

表 5.2: /dev 下常见设备及说明

5.2 文件系统

ISO Image

```
# 创建光盘映像文件 (ISO)
dd if=/dev/cdrom of=cd.iso      # (1)从已有 CD-ROM 光盘创建
genisoimage -o cd-rom.iso -R -J ~/cd-rom-files # (2)从目录生成 ISO 镜像

# 校验 ISO 文件完整性
md5sum /dev/cdrom
md5sum image.iso

# 刻录光盘
wodim dev=/dev/cdrw blank=fast # 擦除可重写光盘 (CD-RW / DVD-RW)
wodim dev=/dev/cdrw image.iso # 刻录 ISO 文件到光盘

# 挂载 ISO 镜像
mkdir /mnt/iso_image
mount -t iso9660 -o loop image.iso /mnt/iso_image
```

Disk Partitioning

```
lsblk      # 列出所有块设备
umount /dev/sdb1 # 操作设备前先卸载 (对分区)

# 分区 (对整个磁盘)
sudo parted /dev/sdb
mklabel gpt
mkpart primary ext4 0% 100%

sudo mkfs -t ext4 /dev/sdb1 # 格式化分区
sudo fsck /dev/sdb1      # 检查并修复文件系统错误
sudo blkid /dev/sdb1     # 查看分区UUID
sudo mount /dev/sdb1     # 挂载分区

# 将配置写入 /etc/fstab
UUID=<uuid> <mount_point> ext4 defaults 0 2
```

Swap 分配

```
free -h
swapon --show
sudo fallocate -l 1G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

6 Shell 脚本

6.1 脚本格式

在 Shell 脚本 的第一行通常会写上 `#!/bin/bash`，这行语句称为 shebang，告诉操作系统在执行脚本时，应该调用哪一个解释器。

6.2 变量

在 Shell 中，变量可以像 Python 一样直接赋值，不过这种方式定义的变量都是全局变量。除了直接赋值，还可以通过 `read` 从标准输入读取用户输入并赋值，一般配合 `echo -n` 给出提示，也可以通过 `$0`、`$1` 等读取命令行参数，`$#` 表示参数的个数，`$@` 表示参数列表。

变量

```
local foo="a"          # 局部变量
echo $REPLY            # read 未指定变量时的默认存储变量
declare -u upper       # 声明变量，赋值时自动转大写
declare -l lower       # 声明变量，赋值时自动转小写
read -p "prompt: " -t 5 var      # 限时5s
read -p "password: " -s var      # -s隐藏输入
read -e -i "default" var        # -e命令补全 -i提供默认值
```

在需要向命令传递输入时，Shell 还提供了 两种重定向方式：here string 和 here document。Here String 使用 `<<<`，可以把一行字符串作为标准输入传递给命令；Here Document 则适合多行输入，它把分隔符之间的所有文本传递给命令的标准输入。

Here Document

```
# 其中 EOF 是分隔符，可以自定义，只要首尾一致即可。
cat <<EOF
这是第一行
这是第二行
EOF
```

6.3 语句

语句

```
[ -e file ]          # 文件存在 (-d目录 -f普通文件 -r可读 -s非空)
[[ str1 >= str2 ]]   # 字符串字母序比较, == 支持 glob 通配符 (* ? [...])
[[ str =~ ^[0-9]+$ ]] # =~ 支持正则表达式
(( INT == 0 ))       # 整数比较

for (( i=0; i<6; i=i+1 )); do
    echo $i
done

$((2#1010))          # 2 进制
$((8#77))             # 8 进制 (0 前缀亦可)
$((16#ff))           # 16 进制 (0x 前缀亦可)
$((2**10))           # 求幂, 其余运算符与 C 语言相同
```

6.4 参数展开

表示从头匹配, % 表示从尾匹配; 双符号 ## / %% 为最长匹配, 单符号为最短匹配。

参数展开

<code>"\${a}_file"</code>	# 变量与字符串拼接
<code>\${a:-"test"}</code>	# a 未设置时用 test 替代
<code>\${a:+test}</code>	# a 已设置时用 test 替代
<code>\${a:=test}</code>	# a 未设置时用 test 替代并同时赋值给 a
<code>\${a:?error}</code>	# a 未设置时返回错误
<code>\${#a}</code>	# 字符串长度
<code>\${a:offset}</code>	# 从 offset 开始的子串
<code>\${a:offset:length}</code>	# 从 offset 开始长度为 length 的子串
<code>\${a,,}</code>	# 转小写
<code>\${a^^}</code>	# 转大写
<code>\${a^}</code>	# 首字母大写
<code>\${a#pattern}</code>	# 从头删除最短匹配
<code>\${a%%pattern}</code>	# 从尾删除最长匹配
<code>\${a/pattern/string}</code>	# 替换第一个匹配
<code>\${a//pattern/string}</code>	# 替换所有匹配
<code>\${a/#pattern/string}</code>	# 替换开头匹配
<code>\${a/%pattern/string}</code>	# 替换结尾匹配

7 系统启动与服务

7.1 cron

通过运行 `crontab -e` 命令来编辑定时任务，文件中每一行表示一个定时任务，格式如下：

cron 任务格式

```
* * * * * /usr/bin/python3 example.py >> output.log 2>&1
| | | | |
| | | | +---- 星期 (0 - 7) (0 和 7 都是星期天)
| | | +----- 月 (1 - 12)
| | +----- 日 (1 - 31)
| +----- 小时 (0 - 23)
+----- 分钟 (0 - 59)
```

时间表达式	含义
0 * * * *	每小时的第 0 分钟执行
0 9 * * *	每天上午 9 点执行
0 9 * * 1-5	每周一到周五上午 9 点执行
*/10 * * * *	每 10 分钟执行一次

表 7.1: 常见的 Cron 时间表达式说明

注意：编辑保存后无需重启 cron 服务，它会立即生效。如果系统宕机，错过的任务不会被补执行。

7.2 systemd

systemd 是现代 Linux 发行版中常用的初始化系统和服务管理器，用于启动和管理系统服务及进程。systemd 使用单个二进制文件 `/usr/bin/systemd` 作为初始化进程 (PID 1)，并通过一系列配置文件来定义和管理服务。这些配置文件通常位于 `/etc/systemd/system/` 目录下，文件扩展名为 `.service`。

```
/etc/systemd/system/rathole.service
```

```
[Unit]
Description=Rathole Server Service
After=network.target

[Service]
Type=simple
Restart=always
RestartSec=5s

ExecStart=/usr/bin/rathole -s /etc/rathole/rathole.toml

[Install]
WantedBy=multi-user.target
```

```
cron 任务格式
```

```
sudo systemctl daemon-reload
sudo systemctl start rathole
sudo systemctl enable rathole
sudo systemctl status rathole
```

7.3 GRUB

GRUB (GRand Unified Bootloader) 是一个引导加载器程序，负责在计算机启动时加载和引导操作系统内核。GRUB 默认配置文件是 `/etc/default/grub`，`/etc/default/grub.d` 目录里面也存放着一些配置。使用 `update-grub` 命令时，会用上述配置生成最终配置文件 `/boot/grub/grub.cfg`。

`uname -r` 查看当前使用内核，要列出当前系统中已安装的 Linux 内核版本，可以使用命令 `dpkg --get-selections | grep linux-image`。

命令输出解释：

- `ii`：表示已安装，并成功配置。
- `rc`：表示已删除，但配置文件仍然存在。

如果你想删除旧内核版本，可以使用命令 `sudo apt purge linux-image-x.x.x-generic`。删除内核后，需要运行 `sudo update-grub` 命令来更新 GRUB 配置。如果你需要清理残留依赖包，运行 `sudo apt autoremove`。

进入 grub:

- 命令行: 开机长按 `esc` 进入命令行。
- 图形界面: 开机长按 `Shift`, 如果不行, 先进入命令行, 运行 `normal` 命令, 之后按一下 `esc`。